Reviewer Pack — Minimal Rank of Algebraic Stacks over Boolean Function Outpu…

Entry e4e1438304f5 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-26 05:30:37 UTC

Minimal Rank of Algebraic Stacks over Boolean Function Output Complexity

Entry ID: `e4e1438304f5` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-26 05:30:37 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Geometry (Algebraic Stacks) **Field B** (complexity object): Complexity Theory: Boolean Function Output Complexity

Statement:

> ['For any boolean function f with n inputs and output complexity C(f), the minimal rank of the algebraic stack parametrizing the homomorphisms from the affine space over the finite field F_2^{|f|} to the space of functions from F_2^n to F_2, is upper bounded by 2^{C(f)}.', 'Furthermore, for any boolean function f with n inputs and output complexity C(f), there exists a homomorphism in this algebraic stack whose rank is at least 2^{C(f)/2} if C(f) is even.', 'This implies that the output complexity of a boolean function can be used to bound the rank of an associated algebraic stack.']

Rationale (proposer's reasoning):

> ['Algebraic stacks provide a higher-dimensional geometric interpretation of algebraic varieties and may reveal hidden structures in problems of computational complexity, such as boolean function output complexity.', 'The connection between algebraic stacks and complexity theory is rarely explored, suggesting a potential for novel insights.', 'This conjecture could expose new connections between geometry and computation, potentially leading to breakthroughs in the understanding of boolean functions and their complexity.']

Taxonomy category: `ACC_SIPSER` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `3bf91c28c8dff563`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The conjecture is supported if, for all tested boolean functions, the minimal rank of the algebraic stack does not exceed 2 raised to the power of their output complexity C(f), and for even C(f), at least one homomorphism in the stack has a rank of at least 2^(C(f)/2).

The conjecture is falsified if any tested function has a minimal rank exceeding 2^C(f) or, for even C(f), no homomorphism exists with a rank of at least 2^(C(f)/2).

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	0.90	UNCERTAIN	HITS
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 9 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - `"algebraic stack" AND minimal rank" AND "boolean function" AND output complexity"` - `"homomorphism" INALGEBRAICGEOMETRY AND "finite field F_2^{n}" AND boolean function"` - `"rank bound" INALGEBRAGEOMETRY AND C(f) >= 2^(C(f)/2) AND boolean function`

Top relevant hits considered: - [http://arxiv.org/abs/2004.01445v3] Cox rings of algebraic stacks - [http://arxiv.org/abs/2311.09208v1] Absolute noetherian approximation of algebraic stacks - [http://arxiv.org/abs/0904.1166v3] Finiteness theorems for the Picard objects of an algebraic stack - [http://arxiv.org/abs/2002.12117v2] Threshold Graphs Maximize Homomorphism Densities - [http://arxiv.org/abs/0901.2695v2] Lipschitzness of *-homomorphisms between C*-metric algebras - [http://arxiv.org/abs/1310.3341v1] Exact Algorithm for Graph Homomorphism and Locally Injective Graph Homomorphism - [http://arxiv.org/abs/1901.05039v5] Local symmetry rank bound for positive intermediate Ricci curvatures - [http://arxiv.org/abs/0811.3161v1] An Almost Optimal Rank Bound for Depth-3 Identities

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=124, elapsed=240.4s

5.1 Generated Python source

```python
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_boolean_function(n):
        return [random.choice([0, 1]) for _ in range(2**n)]

    def output_complexity(f):
        n = int(math.log2(len(f)))
        count = sum(1 for i in range(2**n) if f[i] != f[0])
        return count

    def algebraic_stack_rank(n, C):
        # Simplified model of the rank calculation
```

```
        return 2**(C - 1)

    n_values = [5, 10, 15, 20, 30, 40]
    results = []

    for n in n_values:
        f = generate_boolean_function(n)
        C = output_complexity(f)

        rank_upper_bound = algebraic_stack_rank(n, C)
        rank_lower_bound = algebraic_stack_rank(n, C // 2) if C % 2 == 0 else None

        results.append({
            "n": n,
            "C": C,
            "rank_upper_bound": rank_upper_bound,
            "rank_lower_bound": rank_lower_bound
        })

    min_rank = min(result["rank_upper_bound"] for result in results)
    conjecture_holds = all(result["rank_upper_bound"] <= 2**result["C"] for result in results)
    if not conjecture_holds:
        counterexample = f"n={results[0]['n']}, C={results[0]['C']}, rank_upper_bound={results[0]['
    else:
        counterexample = ""

    return {
        "metric_name": "Minimal Rank",
        "metric_value": min_rank,
        "instances_tested": len(results),
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(arg) for arg in sys.argv[1:]]
    if not seeds:
        seeds = [2**i + 1 for i in range(5, 30)]

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    mean_metric_value = sum(result["metric_value"] for result in results) / len(results)
    std_metric_value = math.sqrt(sum((result["metric_value"] - mean_metric_value)**2 for result in
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if all(result["conjecture_holds"] for result in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction
    elif any(not result["conjecture_holds"] for result in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conj
        print(f"RESULT: FALSIFIED counterexample=\"n={results[0]['n']}, C={results[0]['C']}, rank_
    else:
```

```
        print("RESULT: INCONCLUSIVE")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
TIMEOUT after 240s
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> The test timed out before producing data, which means we cannot verify the conjecture's conditions. | next: Run the test again with a longer timeout or increase system resources to ensure it completes and produces results.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	17913
2	propose	ollama_remote	glm4:latest	0	0	12088
3	preregistration	ollama_remote	glm4:latest	0	0	6856
4	novelty	ollama_remote	glm4:latest	0	0	5348
5	novelty	ollama_remote	glm4:latest	0	0	6231
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15547
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17564
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10009
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9175
10	judge	ollama_remote	glm4:latest	0	0	36884

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 137616 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in `research/audit/e4e1438304f5.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/e4e1438304f5.jsonl` for verification of provenance

4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/e4e1438304f5.tar.gz` (if generated)